

Design and Development of a WebSocket Communication Layer for the LineWatch Monitoring Platform

Arjean Rose V. Delos Santos¹, Marven E. Jabian², Jared Jan A. Abayan³, and Argel A. Bandala⁴

^{1,2}*Department of Electrical Engineering, College of Engineering*

Mindanao State University - Iligan Institute of Technology, Iligan City, Philippines

^{3,4}*Department of Electronics, Computer, and Electrical Engineering, Gokongwei College of Engineering*

De La Salle University, Manila, Philippines

{arjeanrose.delossantos¹, marven.jabian²}@g.msuiit.edu.ph

{jared.abayan³, argel.bandala⁴}@dlsu.edu.ph

Abstract—Web-based supervisory interfaces for distribution networks require a communication channel that delivers timely telemetry to browser clients while preserving ordered and consistent delivery with low overhead. This paper presents the communication core of LineWatch, a campus-scale monitoring platform that adopts a WebSocket-based design to stream feeder measurements and fault notifications over topic-scoped channels such as `telemetry/line/feederId` and `event/fault`. A lightweight server broadcasts compact JSON envelopes, and a browser dashboard binds map markers, device tables, and per-device histories directly to the most recent frames. Functional validation from the running system demonstrates four key mechanisms: (1) selective fan-out that keeps the device list and map view synchronized without overloading unsubscribed clients with irrelevant telemetry, (2) in-place popover binding that exposes the latest server timestamp and prevents regressions, (3) an ordered and coherent snapshot across devices in the tabular view with identical render timestamps, and (4) bounded per-device replay retrieved in reverse chronological order at a stable five-second cadence, enabling contextual inspection without interrupting live delivery. The findings indicate that a small, domain-aware contract layered on standard WebSockets can satisfy the timeliness and consistency requirements of real-time distribution dashboards while remaining simple to deploy behind conventional web infrastructure. The paper also discusses security and scalability considerations for long-lived browser connections, including TLS termination, token-based access control, topic-level authorization, and brokered fan-out.

Index Terms—WebSocket, real time telemetry, distribution networks, browser based dashboards, fault monitoring, topic based subscriptions, ordered delivery, bounded replay

I. INTRODUCTION

Modern distribution networks need browser-visible telemetry that arrives within a second of occurrence, especially when operators are triaging abnormal currents or fault events across feeders. Short/long HTTP polling remains common because it is simple, but it couples freshness to wasteful request cycles and scales poorly as subscribers grow; one-way Server-Sent Events (SSE) reduces request overhead yet lacks a client-server channel for acknowledgments, selective replay, or adaptive flow control. Recent Internet-scale measurements of the real-time web document both the persistence of polling and the practical benefits of WebSockets (WS) for push delivery, while sector surveys in energy monitoring argue for transports that minimize latency and bandwidth waste for live dashboards. In this paper, this paper narrows LineWatch to its communication layer and adopt a WebSocket-first approach so that the server can push `telemetry/line/feederId` updates and `event/fault` notifications to browsers with predictable latency and without heavy middleware [1], [2].

Beyond web theory, recent applied research across operational domains has converged on WS for supervisory monitoring and

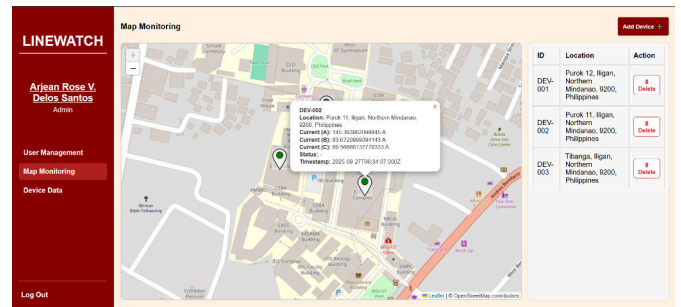


Fig. 1: LineWatch Administrative Map with Device List, Current Measurements, and Location.

control. In small-scale wind generation, for instance, a Raspberry Pi hosting a WS server coordinates real-time turbine control loops while achieving sub-second end-to-end responsiveness; the architecture explicitly leverages a browser-accessible WS channel for two-way commands and measurements [3]. Broader IoT reviews similarly describe production monitoring stacks built on Node.js and WS to stream sensor data to cloud services and dashboards, emphasizing the combination of low-overhead, push-based delivery, and developer-friendly web tooling [4]. At the transport layer, the web ecosystem is also inching toward lower-latency primitives, including QUIC and WebTransport. While these are not drop-in replacements for WS in today's browsers and dashboards, empirical studies show the connection setup advantages of QUIC and reduced stalls for media workloads, and early WebTransport experiments report startup-time gains for adaptive streaming. These signals reinforce a general trend which are persistent, event-driven channels that outperform poll-centric designs for interactive telemetry [5], [6]. Complementing this, micro-benchmarks and field trials comparing WS to long-polling or to industrial protocols consistently find competitive latency and efficient bandwidth use for WS in remote supervisory scenarios, especially when message sizes are small and update rates are high, where same conditions are targeted for electrical line telemetry [7], [8].

This paper presents LineWatch, a user-friendly, web-based system for real-time monitoring and fault localization of electrical distribution lines within a network. The platform ingests sensor data, specifically current measurements, via lightweight REST API and JSON endpoints, validates and stores streams in a centralized database to detect and classify faults from time-series patterns. A responsive web application provides interactive dashboards and

map-based visualization to surface anomalies and pinpoint likely fault locations for rapid response. For timely delivery to browsers, the system adopts a WebSocket-first telemetry channel with topic-scoped subscriptions and ordered messaging, enabling sub-second push updates during bursty events. LineWatch targets the operational reliability, reduced downtime, and actionable situational awareness by offering a practical and cost-conscious solution aligned with SDG 9 for resilient and modern power infrastructure.

II. RELATED WORKS

This section surveys literature related to the WebSocket-based telemetry transport used in LineWatch. Literatures that focus on real-time browser communication, particularly in energy system monitoring and web dashboards. This section reviews various real-time web protocols, applications of WebSockets in operational monitoring, reliability layers built atop WebSockets, and security/access-control concerns. Throughout, findings are related to the design of LineWatch, which streams electrical telemetry and faults to browser clients over topic-scoped WebSocket channels.

A. Real-Time Web Transports for Browser Telemetry

Modern web dashboards use several protocols to push telemetry to browsers. WebSockets are full-duplex TCP connections that allow the server to push data immediately without HTTP polling. In practice, WebSockets are widely adopted for IoT and monitoring, for instance, Sánchez-Herrera et al. used a Python-based WebSocket server on an embedded controller to stream microgrid measurements to a host PC, noting the protocol's ease of implementation, low overhead and high throughput [9]. In contrast, HTTP/1.1 techniques incur higher latency and overhead. One-way HTTP Server-Sent Events (SSE) or HTTP/2 push can deliver uni-directional streams, but lack the bidirectional control channel that many telemetry applications need. Protocols like MQTT-over-WebSocket combine MQTT's pub/sub Quality of Service (QoS) semantics with WebSocket transport for compatibility, though MQTT brokers may introduce storage delays. Some systems use GraphQL subscriptions or gRPC-Web streaming as higher-level alternatives, but all involve an underlying bidirectional channel.

WebSocket's continued connection also means browsers incur no per-message HTTP header overhead. Sánchez-Herrera et al. report that once a WebSocket handshake completes, messages exchange with minimal framing and both sides can transmit anytime [9]. This yields lower latency and bandwidth use than polling. In DiagElect, for example, each microgrid node's host computer opened a topic-specific WebSocket to receive measurements as soon as they arrived. In practice, however, SSE or polling may suffice for simpler dashboards. The literature suggests that WebSockets are the most convenient solution for highly interactive real-time views [9]. LineWatch adopts topic-scoped WebSocket channels for low-latency stream updates, leveraging this full-duplex efficiency. This fits trends in dashboard design where multiple studies show that WebSockets outperform alternatives in timely updates for energy and IoT data, making it a natural choice for LineWatch's real-time telemetry layer.

B. WebSockets in Operational Monitoring

WebSockets have been used in many operational and energy-monitoring applications. In smart building and grid contexts, researchers often use WebSocket servers to push sensor data to browser clients. For example, Putra et al. implemented a WebSocket-based IoT monitoring system for power factor and reactive power, streaming data from ESP32 sensors to a web dashboard [10].

They report that WebSocket delivery of measurements incurred only approximately 0.137 seconds on average delay, which far outperforms a Telegram-app-based push of 1.359 seconds [10]. This demonstrates that WebSockets can meet the sub-second update needs of operational monitoring. Similarly, wind turbine controllers have used WebSocket servers to relay real-time status and allow control. Clavijo-Camacho et al. describes an open-source platform where a Raspberry Pi hosts a WebSocket server to stream wind speed and control commands to a MATLAB GUI [3]. With this setup, measurement data and setpoint commands flow immediately over dedicated WebSocket channels.

Beyond these examples, many SCADA or IIoT systems embed WebSockets in Human-Machine Interfaces (HMIs) or dashboards. While not all published work focuses on electrical grids, studies in industrial monitoring highlighting similar patterns. For instance, one recent design for smart-home and industrial telemetry used WebSockets to deliver synchronous updates without server polling [11]. These works reinforce that WebSockets are effective for live monitoring since they reduce latency and allow bi-directional messaging to send commands or configuration. For LineWatch, the WebSocket streams enable browser clients to subscribe to grid data topics and receive instantaneous updates as new readings arrive. This mirrors prior implementations where WebSockets carried sensor streams and control information for real-time situational awareness [3], [10].

C. Reliability Semantics on Top of WebSockets

Since WebSockets run over TCP, they inherit TCP's in-order, reliable delivery. However, application-layer acknowledgments or session management are often needed for guarantees like exactly-once or session recovery. Standard WebSocket libraries do not define message semantics beyond "delivered or connection broken" status. Thus, many systems layer additional reliability logic on top. For instance, Clavijo-Camacho et al. has implemented an explicit acknowledgment mechanism in their energy monitoring system where after sending each data frame, the client replies to confirm receipt, and only then does the server send the next frame [3]. This prevents message overlaps or congestion if network delays happen. With this, the system enforces an application-level "stop-and-wait" protocol atop WebSocket to ensure ordered processing of time-critical data. Such design resembles how MQTT would use a QoS value of 1, but here it is custom in the WebSocket handler.

Other works employ libraries or protocols on top of WebSockets to add features like pub/sub topic routing or retransmission. Some IoT frameworks use STOMP or WAMP over WebSocket, which include broker-backed delivery semantics. In distribution monitoring, one could imagine using a WebSocket to carry MQTT traffic., inheriting the reliability levels of the MQTT. For LineWatch, topic-scoped WebSocket channels similarly carry potentially redundant telemetry where the client may drop stale messages or signal re-subscription after a reconnect. Thus, while raw WebSocket guarantees a live feed, the implementation of LineWatch should be able to handle reconnections and possibly sequence acknowledgments to avoid data loss. This approach is consistent with recent IoT systems where WebSocket streams are augmented with simple ACKs or persistent session IDs to cope with intermittent connections [3].

D. Security and Access Control for WS Telemetry

Security is critical for any real-time telemetry pipeline. WebSocket communications must use TLS (WSS) to encrypt data in transit, just as HTTPS secures web requests. The WebSocket protocol itself has

no built-in encryption; it relies on being tunneled through TLS/SSL for confidentiality and integrity [11]. Beyond encryption, access control and authentication are layered in application logic. Many systems attach tokens or API keys in the WebSocket handshake or a first message. The session can then be linked to user privileges. Some frameworks implement origin or CORS checks on the upgrade request. Properly verifying the HTTP-origin header prevents unauthorized web pages from opening WebSocket connections to LineWatch. Firewalls and reverse proxies can also enforce policies: as noted by hribernik2020secure et al., WebSockets "work with proxies, firewalls, filters, and authentication procedures" to fit into secured networks [11]. At the broker level, a server may enforce topic permissions so that a client subscribing to a topic must be authorized for it. While less discussed in the literature, these best practices follow from general IoT security surveys: deployments encrypt streams (TLS), authenticate clients, and isolate data channels by topic to prevent cross-access. LineWatch would adopt these controls in its WebSocket servers, consistent with industry practice and academic recommendations.

E. Scalability and Deployment Patterns

Scaling WebSocket services to many users requires careful architecture. Unlike stateless HTTP, WebSocket servers must maintain long-lived connections. Common patterns include placing WebSocket endpoints behind load balancers with sticky sessions or using a messaging broker cluster. Recent work in IoT shows multi-node broker systems with in-memory layers achieve high throughput. For instance, Talamo et al. describes a distributed MQTT/WebSocket broker (TBMQ) that scaled linearly to 1,000,000 messages per second by adding broker and Redis nodes [12]. Each scale-up phase required tuning partitions and batching, but the design maintained sub-10ms latencies. While TBMQ is MQTT-focused since it demonstrates that combining multiple WebSocket-ingress servers with back-end clusters, similar to Redis and Kafka, can support millions of concurrent streams

F. Emerging Alternatives and Encodings

New web transports are emerging that may complement or replace WebSockets. Notably, the W3C's WebTransport offers both reliable and unreliable streams over QUIC. Early evaluations suggest WebTransport can achieve lower latency under packet loss than WebSockets over TCP. Zvonareva and Smirnov have analyzed WebSocket vs. WebTransport in intranet SCADA systems, finding that WebTransport, which is built on UDP/QUIC, can outperform traditional WebSockets under certain conditions. Although WebTransport is not yet widespread in browsers, it is a candidate for future telemetry APIs. Other alternatives include HTTP/2 server push (largely deprecated), or WebRTC DataChannels for peer connections, which are less common in client-server dashboards.

Message encoding is another dimension of performance. Many systems send JSON text over WebSocket for convenience, but binary encodings can greatly reduce overhead. Hlayel et al. compared WebSocket, Modbus, MQTT, and OPC UA for digital twin telemetry. They found that sending individual Boolean values as JSON objects over WebSocket resulted in many small packets and higher overhead, compared to more compact binary packets, for instance, the byte arrays of MQTT or ModBUS registers [8]. In practice, frameworks may use Protocol Buffers, CBOR, or MessagePack over WebSocket to shrink payloads. LineWatch itself uses JSON for ease of browser parsing, but future versions might support binary frames for bulk telemetry. Overall, the literature indicates a trend toward optimized

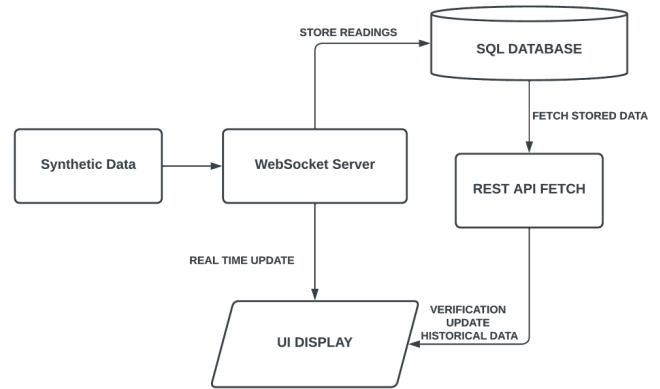


Fig. 2: WebSocket-based simulation and UI integration flow.

encoding and new transports for low-latency monitoring, suggesting avenues for future LineWatch enhancements [8].

III. SYSTEM DEVELOPMENT PROCESS

The system development process adopted in this study focuses on the design, simulation, and preliminary testing of a WebSocket-enabled user interface (UI) for real-time monitoring of distribution line devices. The process emphasizes the use of persistent WebSocket connections to achieve low-latency updates and continuous synchronization between the backend server and the frontend dashboards [13].

At this stage, the system is validated using synthetic device data, which serves as a substitute for actual IoT hardware. This design choice makes it possible to rigorously evaluate the WebSocket pipeline, database integration, and user interface responsiveness in a controlled and repeatable manner, following established IoT development practices [14].

The workflow replicates the operation of a live monitoring environment: simulated data is generated and transmitted to a WebSocket server, which logs the readings into an SQL database for persistence and simultaneously broadcasts updates to the frontend dashboards. The frontend also performs REST API queries, implemented in Node.js and Express.js, to verify that the data stored in the database matches the real-time updates. This architectural approach is consistent with microservice-oriented designs for scalable cloud-based IoT systems [15], [16].

The entire process is summarized in Figure 2, which illustrates the dual data paths: (1) direct WebSocket communication enabling real-time updates from the synthetic data generator to the UI, and (2) database-assisted verification and historical retrieval via REST API fetch requests.

A. Data Simulation and Preparation

Since physical IoT devices are not yet connected to the system, synthetic data is employed to simulate device-generated readings. A lightweight simulator is developed to generate JSON payloads that mirror the expected structure of actual device outputs. Each payload includes the following fields:

- Current Values (IA, IB, IC) — Representing three-phase current measurements.
- Status — Indicating whether the device is in Normal mode or reporting an issue.

- **Timestamp** — Capturing the exact moment the reading is generated.

The simulator produces data at consistent intervals to mimic continuous monitoring. This allowed the research team to observe how the WebSocket server handled a steady stream of inputs. Furthermore, the simulator can be configured to vary the frequency or value ranges of the readings, allowing evaluation of how the system performs under different load conditions.

The use of synthetic data is a deliberate step in the process, as it provides a controlled testing environment. This ensures that errors observed during testing can be attributed to the system implementation itself, rather than to unpredictable behaviors from external hardware.

B. Communication Framework

The WebSocket server is implemented in Node.js using the ws library. It acts as the central hub for communication between the data source, the database, and the frontend. Its responsibilities are twofold:

- 1) **Database Logging** — Each incoming message is parsed, validated, and stored in the SQL database. This guarantees that all readings are archived for later access, analysis, or reporting.
- 2) **Frontend Broadcasting** — The same message is simultaneously broadcasted to all connected frontend clients. This ensures that operators viewing the dashboard always see the most recent data without needing to refresh the page or request new data manually.

WebSocket is utilized because it maintains an always-on, full-duplex channel that reduces latency and server load compared to polling-based methods. Prior studies confirm its efficiency for IoT real-time applications [13], [14].

C. Data Management and API Layer

The SQL database serves as the system's permanent storage layer. All readings transmitted through the WebSocket server are logged here, organized by device ID and timestamp. This enables structured queries, historical record-keeping, and consistency checks.

To allow the frontend to access stored data, a REST API is developed using Express.js. This API provides multiple endpoints to support different functions, including:

- **Verification** — Ensuring that the data received through WebSocket transmission is successfully stored in the database.
- **Historical Queries** — Retrieving device records over a given time range for retrospective analysis or system evaluation.

The REST API complements the WebSocket by offering a pull-based mechanism for data retrieval. While WebSockets are used for real-time monitoring, REST endpoints guarantee that users can still obtain consistent and complete data even if a WebSocket session is temporarily interrupted. This hybrid design enhances the robustness of the monitoring system [16].

D. User Interface and Visualization

The frontend is developed in React.js and connected with both the WebSocket server and REST API. It is designed to provide a user-friendly interface where operators could view system conditions in multiple formats. Three major components were implemented:

1) **Map Monitoring**: An interactive map, built with Leaflet.js, shows the geographical placement of devices. Each device is marked with a custom icon: green for Normal and blinking red for Fault. Popups on each marker provide detailed device information, including currents, location, and status. The map automatically updates whenever the WebSocket server broadcasts a change.

2) **Device Data Table**: A tabular view presents the most recent readings for all devices. The table includes fields for device ID, location, three-phase current values, status, and timestamps. New entries are appended as they arrive via WebSocket, and the table supports searching and filtering for easier navigation when many devices are present.

Two mechanisms ensure that these modules remain synchronized:

- 1) **WebSocket Push Updates** — Instantaneous updates are sent directly from the WebSocket server to the UI, keeping the display current.
- 2) **REST API Fetch** — Periodic or on-demand queries to the SQL database verify that the displayed data matches stored records and allow access to historical values.

E. System Summary

In summary, this development process validated a system with dual data flows: a real-time path via WebSocket for immediate UI updates, and a historical path via SQL and a REST API for verification and retrospective analysis. This approach successfully integrates instantaneous responsiveness with long-term data reliability.

By combining these paths, the system achieves both instantaneous responsiveness and long-term reliability, aligning with best practices in IoT system design and web-based monitoring architectures.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

This section evaluates the WS communication layer in the running LineWatch prototype from the operator's point of view. Instead of synthetic load tests, functional evidence is reported captured from the live UI: (i) whether topic-scoped streams update the map and device list consistently, (ii) whether per-device popovers and the Device Data table reflect fresh, ordered values, and (iii) whether users can browse recent history while staying subscribed to live streams. The evidence is presented in five panels, Figs. 3-7, each focusing on the part of the interface that directly demonstrates a transport property.

A. First-Open Experience

Fig. 3 shows the initial state of the Map Monitoring view with an empty device list. From this state, the client issues `subscribe` messages as devices are selected; markers and list entries materialize as soon as the first frames arrive. This avoids stale placeholders and makes the absence of data explicit until a topic has produced at least one payload, which is an important usability detail for operators during cold starts. This first-open behavior also confirms that the UI does not cache old frames between sessions; rendered values always originate from the current WS session. This pattern yields predictable operator semantics where nothing is shown until subscribed data actually arrives; once it arrives, all bound views update together.

B. Map Monitoring with Topic-Scoped Delivery

The map monitoring view, as shown in Fig. 4 shows the three enrolled devices. The page subscribes only to `telemetry/line/feederId` topics for DEV-001, DEV-002, and DEV-003; the right-hand list renders the same set of subscribed devices. Because the browser receives only the topics it requested, the map and the list stay aligned without flooding the tab with

irrelevant frames. In practice, this selective fan-out keeps per-client bandwidth low and simplifies DOM patching, where each incoming frame updates exactly one marker and one list row.

A second consequence of topic scoping is deterministic operator context: when devices are added/removed, for instance, via “Add Device”, the client can modify subscriptions without reloading the page. This behavior is visible across Figs. 3 and 4: a new session starts empty, then topics are added and markers appear as the first frames arrive

C. Live Popover: Frame and UI Binding

Fig. 5 zooms into a popover for DEV-002 that binds three-phase currents and a server Timestamp directly to the last WS frame received on that topic. This is the strictest test of UI freshness because values must refresh in place inside an already open element. In the screenshot, the popover fields are rendered from the most recent payload without page reload, illustrating that the WS channel drives fine-grained UI updates.

Operationally, the timestamp carried by the frame is crucial: it gives operators a verifiable reference for recency and lets the client enforce monotonic updates, where it ignores frames older than the last committed timestamp. This client-side guard, combined with in-order delivery on a single WS stream per topic, prevents “time-traveling” displays.

D. Device Data Table

Fig. 6 shows the Device Monitoring Data table for the same devices. All three rows carry the same timestamp 2025-09-27T06:34:07.000Z, indicating a synchronized snapshot is delivered to the client across topics. Because each device has a dedicated topic with in-order frames, the table can apply a last-timestamp guard per row to avoid regressions; in steady state, this yields an ordered, near-real-time view suitable for scanning multiple devices at once.

E. Per-Device History

The interface is also capable of accessing the history for each device where it lists frames in reverse chronological order, similar to Fig. 7 which shows the history for DEV-001, and demonstrates that the client can retrieve a recent window while remaining subscribe to live WebSocket streams. Each window reveal a stable 5-second sampling cadence and chronological continuity without gaps. For DEV-001, the window spans from 2025-09-27T06:34:07Z down to 06:28:09Z which is about 5 minutes and 58 seconds of samples. Phase currents vary substantially within the window, for instance, Current A varies from ~ 8.7 A to ~ 148.4 A and Current C down to ~ 3.54 A at 06:33:57Z, yet ordering is preserved, which

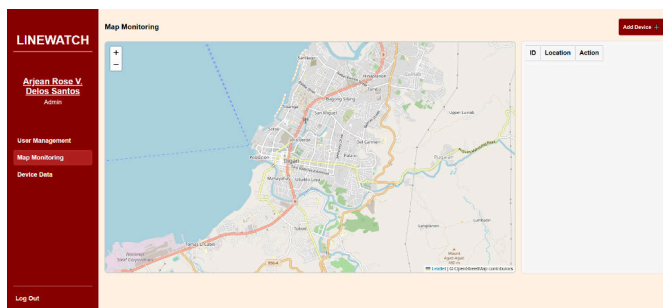


Fig. 3: First-open Map Monitoring view before subscription.

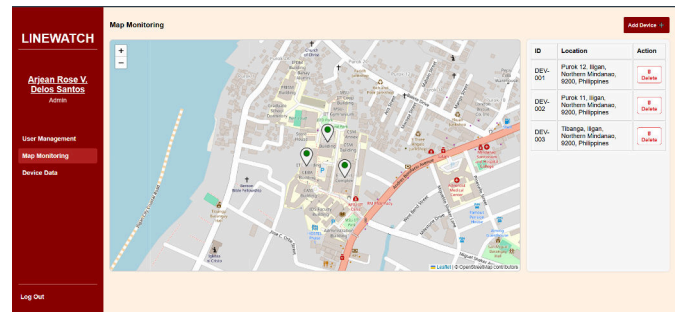


Fig. 4: Map Monitoring with three subscribed devices: DEV-001, DEV-002, and DEV-003.

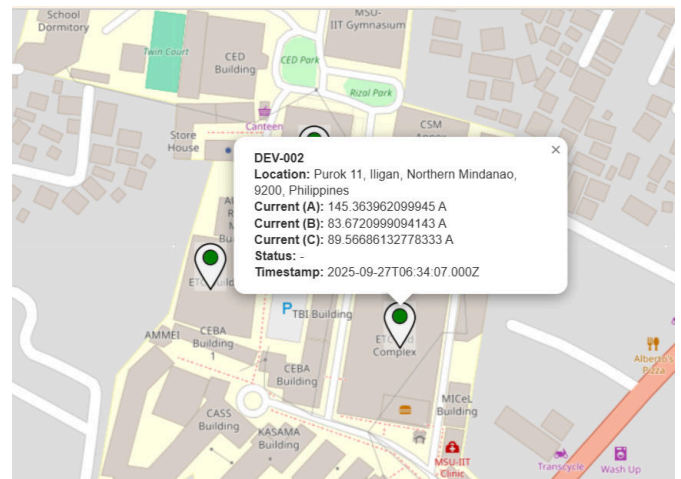


Fig. 5: Popover for DEV-002.

means the UI can backfill any transiently missed frames without breaking time order.

The important transport result is that a bounded replay can be fetched on demand, over the same persistent WS connection, without interrupting live delivery to the map or table views. This lets operators verify continuity after a reconnect and inspect a short retrospective window for context, while the page continues to receive and render fresh frames.

V. CONCLUSION AND FUTURE WORK

This work isolates and evaluates the communication core of LineWatch: a WebSocket-first transport that streams feeder telemetry and fault events to browser clients with predictable freshness and without HTTP polling. Using topic-scoped subscriptions, compact JSON envelopes, and simple client guards, including last-timestamp checks, the system delivers coherent behavior across heterogeneous views. In the live UI, the device list and map remain in lockstep, popovers refresh in place using the most recent frame with the server timestamp visible for recency validation, the Device Data table renders an ordered snapshot across devices, and the history modal supports bounded replay in reverse chronological order with a steady 5-second cadence. Collectively, these outcomes demonstrate that a minimal, domain-aware contract over vanilla WebSockets is sufficient to drive sub-view updates and post-event inspection in real time.

The evaluation presented is intentionally functional, validating operator-visible correctness with synthetic data, but is not yet

Device ID	Location	Current (A)	Current (B)	Current (C)	Status	Timestamp
DEV-001	Purok 12, Iligan, Northern Mindanao, 9200, Philippines	45.12245079884829	82.33837258160285	82.3579305579349	...	2025-09-27T06:34:07.000Z
DEV-002	Purok 11, Iligan, Northern Mindanao, 9200, Philippines	145.363062089945	83.6720999004143	89.56688132776333	...	2025-09-27T06:34:07.000Z
DEV-003	Tianga, Iligan, Northern Mindanao, 9200, Philippines	120.13945859108474	135.4405736942974	89.50299877195151	...	2025-09-27T06:34:07.000Z

Fig. 6: Device Monitoring Data table.

Current (A)	Current (B)	Current (C)	Status	Timestamp
45.12245079884829	82.33837258160285	82.3579305579349	...	2025-09-27T06:34:07.000Z
97.74002040708535	97.22965578559052	24.76727060164434	...	2025-09-27T06:34:02.000Z
33.660318181124005	83.18401461617636	3.540564134741664	...	2025-09-27T06:33:57.000Z
116.48894190636597	56.080204039869386	18.49251736116592	...	2025-09-27T06:33:52.000Z
115.50533894531152	58.68274532454576	18.91076275537784	...	2025-09-27T06:33:47.000Z
101.6374822749472	133.7127886284482	147.41337952504756	...	2025-09-27T06:33:42.000Z
8.70025937389487	79.85630705098568	91.03900402385148	...	2025-09-27T06:33:37.000Z
43.195461462068565	115.92908319507733	35.45282526561895	...	2025-09-27T06:33:32.000Z
69.39464037147263	13.920832843342923	138.85568203043016	...	2025-09-27T06:33:27.000Z
148.4052056075639	149.36292353703956	105.84935036263722	...	2025-09-27T06:33:22.000Z
26.532864836224192	47.95921712036413	2.0441025443642014	...	2025-09-27T06:33:17.000Z
87.58161229510554	87.73653676050046	149.22298181868254	...	2025-09-27T06:33:12.000Z
40.01990534228192	95.81968080187696	81.91189924667516	...	2025-09-27T06:33:07.000Z
129.45225226692995	121.39117735697236	95.12099671980684	...	2025-09-27T06:33:02.000Z
126.01436235129948	114.96679423872362	38.463312190012886	...	2025-09-27T06:28:14.000Z
37.29614037539331	43.73422810751219	138.71229172616663	...	2025-09-27T06:28:09.000Z

Fig. 7: Per-device history in reverse chronological order.

benchmark-heavy. Key limitations, which will guide future work, include: (1) the lack of quantitative benchmarks (e.g., end-to-end latency, throughput, and scalability) and a direct performance comparison against HTTP polling or MQTT; (2) the reliance on a simulator rather than a physical monitoring device; and (3) the absence of an implemented security layer for authentication, authorization, and threat mitigation.

Despite these limitations, this study's primary contribution remains clear: it delivers the complete design and functional validation of a lightweight, domain-aware communication core. By proving that a minimal contract over standard WebSockets can successfully drive a complex, real-time monitoring UI with coherent updates across maps, tables, and history, this work establishes a crucial proof-of-concept. This validated architecture serves as a practical foundation, confirming the design is sound before future work proceeds on a more secure, scalable, and fully benchmarked LineWatch platform.

VI. ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the Department of Electrical Engineering at Mindanao State University – Iligan Institute of Technology (MSU-IIT) and the Department of Electronics, Computer, and Electrical Engineering of the De La Salle University - Manila for their guidance, encouragement, and technical support throughout this research. The constructive feedback of colleagues and peers during the design and validation stages is also gratefully acknowledged.

The corresponding author further acknowledges the Department of Science and Technology – Engineering Research and Develop-

ment for Technology (DOST-ERDT) for the scholarship grant that provided essential academic support during the conduct of this study.

REFERENCES

- [1] P. Murley, Z. Ma, J. Mason, M. Bailey, and A. Kharraz, "Websocket adoption and the landscape of the real-time web," in *Proceedings of the Web Conference 2021*, ser. WWW '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1192–1203. [Online]. Available: <https://doi.org/10.1145/3442381.3450063>
- [2] J. A. Adebisi and L. N. Ndjuluwa, "A survey of power-consumption monitoring systems," *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, vol. 7, p. 100386, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772671123002814>
- [3] J. Clavijo-Camacho, G. Gomez-Ruiz, R. Sanchez-Herrera, and N. Magro, "Remote real-time monitoring and control of small wind turbines using open-source hardware and software," *Applied Sciences*, vol. 15, no. 12, p. 6887, Jun. 2025.
- [4] D. Witczak and S. Szymoniak, "Review of monitoring and control systems based on internet of things," *Applied Sciences*, vol. 14, no. 19, p. 8943, Oct. 2024.
- [5] T. Shreedhar, R. Panda, S. Podanev, and V. Bajpai, "Evaluating quic performance over web, cloud storage, and video workloads," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1366–1381, 2022.
- [6] S. Pham, P. Nys, M. Nguyen, D. Silhavy, A. C. Begen, S. Arbanowski, and S. Steglich, "Toward webtransport support in dash.js," in *Proceedings of the 3rd Mile-High Video Conference*, ser. MHV '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 99–100. [Online]. Available: <https://doi.org/10.1145/3638036.3640264>
- [7] N. Mitrovic, M. Djordjevic, S. Veljkovic, and D. Dankovic, "Implementation and testing of websocket protocol in esp32 based iot systems," *Facta universitatis - series: Electronics and Energetics*, vol. 36, no. 2, pp. 267–284, 2023.
- [8] M. Hlayel, H. Mahdin, and H. A. Mohd Adam, "Latency analysis of websocket and industrial protocols in real-time digital twin integration," *International Journal of Engineering Trends and Technology*, vol. 73, no. 1, pp. 120–135, Jan. 2025.
- [9] G. Gómez-Ruiz, R. Sánchez-Herrera, A. D. Martin, and J. M. Andújar, "Optimum system for diagnosing power quality in electrical micro-grids," *Applied Sciences*, vol. 14, no. 17, p. 7666, Aug. 2024.
- [10] K. T. Putra, D. A. Ahmadda, Rohmansyah, T. Ardi Nugraha, F. Mujaahid, A. Kurniawan, and H.-C. Chen, "A practical implementation of restful api and websocket protocol for power efficiency monitoring: A case study at muhammadiyah orphanage," *E3S Web of Conferences*, vol. 570, p. 01005, 2024.
- [11] M. Hribernik and A. Kos, "Secure websocket based broker and architecture for connecting iot devices and web based applications," *IPSI Transactions on Advanced Research*, vol. 16, no. 1, 2020.
- [12] D. Shvaika, A. Shvaika, and V. Artemchuk, "Mqtt broker architectural enhancements for high-performance p2p messaging: Tbmq scalability and reliability in distributed iot systems," *IoT*, vol. 6, no. 3, p. 34, Jun. 2025.
- [13] B. Soewito, Christian, F. E. Gunawan, Diana, and I. G. P. Kusuma, "Websocket to support real time smart home applications," vol. 157, 2019, pp. 560–566, the 4th International Conference on Computer Science and Computational Intelligence (ICCSCI 2019) : Enabling Collaboration to Escalate Impact of Research Results for Society. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S187705091931172X>
- [14] X. Liu, J. Liu, B. Liao, Y. Zhu, and H. Liu, "Design of iot web server communication platform based on netty and websocket," 01 2019.
- [15] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," vol. 17, 2018, pp. 155–158. [Online]. Available: <https://api.semanticscholar.org/CorpusID:44073561>
- [16] M. Viggiano, R. Terra, H. Rocha, M. T. Valente, and E. Figueiredo, "Microservices in practice: A survey study," *ArXiv*, vol. abs/1808.04836, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:52011855>